

2026 CCF 非专业级别软件能力认证第一轮
(CSP-S1) 提高级 C++语言试题

认证时间：2026 年 9 月 31 日 14:30~16:30

考生注意事项：

- 试题纸共有 20 页，答题纸共有 1 页，满分 100 分。请在答题纸上作答，写在试题纸上的一律无效。
- 不得使用任何电子设备（如计算器、手机、电子词典等）或查阅任何书籍资料。

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 在 Linux 系统中，如果你想显示当前工作目录的路径，应该使用哪个命令？（ ）
A. pwd
B. cd
C. ls
D. echo
2. 二进制数 10111010 与 01101101 进行按位与运算，其结果是（ ）
A. 00101001
B. 00101000
C. 00011000
D. 11111111
3. 已知 $T(1)=1$ ，且对 $n \geq 2$ 有 $T(n)=T(n/2)+1$ ，则 $T(n)$ 的数量级为（ ）
A. $O(n)$
B. $O(\log n)$
C. $O(n \log n)$
D. $O(n^2)$
4. 6 名同学围成一圈开会，旋转后相同的坐法视为同一种坐法，则不同的坐法共有（ ）
A. 60 种
B. 120 种
C. 240 种
D. 720 种
5. 中缀表达式 $(a+b)*c$ 对应的后缀表达式为（ ）
A. $ab+c*$
B. $ab*c+$
C. $abc*+$
D. $+ab*c$
6. 具有 n 个顶点的完全图（任意两个不同顶点之间恰有一条边）共有（ ）条边

- A. $n(n-1)$
 - B. $n(n-1)/2$
 - C. n^2
 - D. $n(n+1)/2$
7. 深度为 h (根的深度为 1) 的完全二叉树, 其结点个数最多为 ()
- A. 2^h-1
 - B. $2^{(h-1)}$
 - C. 2^h
 - D. 2^{h+1}
8. 对已经有序的数组执行快速排序, 若每次划分都选择第一个元素作为基准元素, 则其最坏时间复杂度为 ()
- A. $O(n \log n)$
 - B. $O(n^2)$
 - C. $O(n)$
 - D. $O(\log n)$
9. 已知函数 `int lowbit(int x) { return x & (-x); }`, 则 `lowbit(10)` 的值为 ()
- A. 1
 - B. 2
 - C. 4
 - D. 8
10. 设哈希表长为 7, 哈希函数为 $h(x) = x \bmod 7$, 采用线性探查解决冲突。依次插入 10、17、31 的过程中, 共发生 () 次冲突
- A. 1
 - B. 2
 - C. 3
 - D. 4
11. 用 4 对括号可以组成的不同合法括号序列共有 () 个
- A. 8
 - B. 10
 - C. 12
 - D. 14
12. 关于 Dijkstra 算法, 下列说法正确的是 ()
- A. 可以处理含负权边的图
 - B. 要求图中所有边权均为非负数
 - C. 可以处理含负权环的图
 - D. 使用二叉堆优化后, 时间复杂度仍为 $O(n^2)$
13. 整数 a 在模 m 意义下存在乘法逆元的充要条件是 ()
- A. m 是质数

- B. $a < m$
 - C. a 与 m 互质
 - D. a 是质数
14. 一个有向图存在拓扑排序的充要条件是 ()
- A. 该图为有向无环图
 - B. 该图为强连通图
 - C. 图中无重边
 - D. 图中每个顶点的入度均不超过 1
15. 在 1 到 100 的整数中, 不能被 2、3、5 中任何一个整除的数共有 () 个
- A. 26
 - B. 24
 - C. 28
 - D. 32

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填 √，错误填 ×；除特殊说明外，判断题 1.5 分，选择题 3 分，共计 40 分）

(1)

```
0101 #include <bits/stdc++.h>
0202 using namespace std;
0303 const int M = 2e6 + 5;
0404 long long A[200005], B[200005], pA[200005], pB
    [200005], F[M], G[M];
0505 int n, cx, cy;
0606 long long D;
0707 long long C(long long *a, long long *p, long long x)
    {
0808     int k = lower_bound(a, a + n, x) - a;
0909     long long L = k > 0 ? p[k - 1] : 0, s = 0;
1010     if (k > 0)
1111         s += (long long)k * x - L;
1212     if (k < n)
1313         s += p[n - 1] - L - (long long)(n - k) * x;
1414     return s;
1515 }
1616 int main() {
1717     ios::sync_with_stdio(false);
1818     cin.tie(0);
1919     cin >> n >> D;
2020     for (int i = 0; i < n; i++)
2121         cin >> A[i] >> B[i];
2222     sort(A, A + n);
2323     sort(B, B + n);
2424     pA[0] = A[0];
2525     pB[0] = B[0];
2626     for (int i = 1; i < n; i++)
2727         pA[i] = pA[i - 1] + A[i], pB[i] = pB[i - 1]
+ B[i];
2828     for (long long x = A[n - 1] - D; x <= A[0] + D;
x++) {
```

2929	long long v = C(A, pA, x);
3030	if (v <= D)
3131	F[cx++] = v;
3232	}
3333	for (long long y = B[n - 1] - D; y <= B[0] + D;
	y++) {
3434	long long v = C(B, pB, y);
3535	if (v <= D)
3636	G[cy++] = v;
3737	}
3838	sort(F, F + cx);
3939	sort(G, G + cy);
4040	long long ans = 0;
4141	for (int i = 0, j = cy - 1; i < cx; i++) {
4242	while (j >= 0 && F[i] + G[j] > D)
4343	j--;
4444	if (j < 0)
4545	break;
4646	ans += j + 1;
4747	}
4848	cout << ans;
4949	}

判断题

- 16. 函数 C 的功能是计算所有 $|A[i]-x|$ 的和，即数组 A 中所有元素与 x 的距离之和。()
 - A. 正确
 - B. 错误
- 17. 若将枚举 x 的循环改为从 $A[0]-D$ 到 $A[n-1]+D$ (其余部分不变)，程序的输出一定发生改变。()
 - A. 正确
 - B. 错误
- 18. 当 $x < A[0]-D$ 时，不可能有满足 $\text{sum}(|A[i]-x|) \leq D$ 的 x，因此枚举 x 从 $A[n-1]-D$ 开始是充分的。()
 - A. 正确
 - B. 错误

单选题

- 19. 函数 C 中 $k = \text{lower_bound}(a, a+n, x) - a$ 表示 ()
 - A. a 中小于 x 的元素个数
 - B. a 中不大于 x 的元素个数
 - C. a 中等于 x 的元素个数
 - D. a 中大于 x 的元素个数
- 20. 当输入为
 - 2 3
 - 0 0
 - 1 0时, 程序的输出为 ()
 - A. 6
 - B. 7
 - C. 8
 - D. 10

(2)

```
0101 #include <bits/stdc++.h>
0202 using namespace std;
0303 const int N = 6e2 + 5;
0404 int g[N][N], w[N], v[N], id[N], n, m;
0505 int main() {
0606     ios::sync_with_stdio(0);
0707     cin.tie(0);
0808     cin >> n >> m;
0909     for (int i = 1; i <= n; ++i)
1010         id[i] = i;
1111     for (int i = 0, x, y, z; i < m; ++i) {
1212         cin >> x >> y >> z;
1313         g[x][y] += z;
1414         g[y][x] += z;
1515     }
1616     int ans = n < 2 ? 0 : 2e9, cur = n;
1717     while (cur > 1) {
1818         memset(w, 0, sizeof(w));
1919         memset(v, 0, sizeof(v));
2020         int p = 0, u = 0;
2121         for (int i = 1; i <= cur; ++i) {
2222             u = -1;
2323             for (int j = 1; j <= cur; ++j)
2424                 if (!v[id[j]] && (u == -1 || w[id[j]
]] > w[u]))
2525                     u = id[j];
2626             v[u] = 1;
2727             if (i == cur) {
2828                 if (w[u] < ans)
2929                     ans = w[u];
3030                 for (int j = 1; j <= cur; ++j) {
3131                     int t = id[j];
3232                     if (t != u)
3333                         g[p][t] += g[u][t], g[t][p]
```

	<code>+= g[t][u];</code>
3434	<code>}</code>
3535	<code>for (int j = 1; j <= cur; ++j)</code>
3636	<code>if (id[j] == u) {</code>
3737	<code>id[j] = id[cur];</code>
3838	<code>break;</code>
3939	<code>}</code>
4040	<code>--cur;</code>
4141	<code>} else {</code>
4242	<code>p = u;</code>
4343	<code>for (int j = 1; j <= cur; ++j)</code>
4444	<code>if (!v[id[j]])</code>
4545	<code>w[id[j]] += g[u][id[j]];</code>
4646	<code>}</code>
4747	<code>}</code>
4848	<code>}</code>
4949	<code>cout << ans;</code>
5050	<code>}</code>

判断题

- 21. 该程序求出的是无向连通图的全局最小割（删去后使图不再连通的最小边权和）。()
 - A. 正确
 - B. 错误
- 22. 当输入的 $n=1$ 时，程序的输出为 0。()
 - A. 正确
 - B. 错误
- 23. 主循环 `while (cur > 1)` 恰好执行 n 轮。()
 - A. 正确
 - B. 错误

单选题

- 24. 在每一轮循环中，最后被加入集合的顶点 u 对应的 $w[u]$ 表示 ()
 - A. u 与所有未加入集合的顶点之间的边权和
 - B. u 与已加入集合的顶点之间的边权和（即当前划分的割的大小）
 - C. u 的度数
 - D. u 与上一个加入集合的顶点之间的边权和
- 25. 该算法的时间复杂度为 ()
 - A. $O(n^2)$

B. $O(n^3)$

C. $O(nm)$

D. $O(m \log n)$

● 26. 当输入为

4 6

1 2 5

1 3 1

2 4 1

3 4 2

2 3 1

1 4 2

时，程序的输出为（ ）

A. 3

B. 4

C. 5

D. 6

(3)

```
0101 #include <bits/stdc++.h>
0202 using namespace std;
0303
0404 const int N = 8e5 + 5;
0505
0606 struct Q {
0707     int t, op, x, y, id;
0808     long long v;
0909 } q[N], p[N];
1010
1111 int n, m, tot, qc, a[N];
1212 long long c[N], ans[N];
1313 set<int> S[N];
1414
1515 void add(int x, long long v) {
1616     for (; x; x -= x & -x)
1717         c[x] += v;
1818 }
1919
2020 long long ask(int x) {
2121     long long s = 0;
2222     for (; x < N; x += x & -x)
2323         s += c[x];
2424     return s;
2525 }
2626
2727 void add_e(int u, int v, int w) {
2828     if (u && v)
2929         q[++tot] = {tot, 0, v, u, 0, 1LL * w * (v -
3030 u)};
3131 }
3232 void cdq(int l, int r) {
3333     if (l >= r)
```

```

3434         return;
3535     int m = (l + r) / 2;
3636     cdq(l, m);
3737     cdq(m + 1, r);
3838     int j = l;
3939     for (int i = m + 1; i <= r; ++i) {
4040         if (q[i].op) {
4141             while (j <= m && q[j].x <= q[i].x) {
4242                 if (!q[j].op)
4343                     add(q[j].y, q[j].v);
4444                 j++;
4545             }
4646             ans[q[i].id] += ask(q[i].y);
4747         }
4848     }
4949     for (int k = l; k < j; ++k)
5050         if (!q[k].op)
5151             add(q[k].y, -q[k].v);
5252     int i = l, k = l;
5353     j = m + 1;
5454     while (i <= m && j <= r)
5555         p[k++] = (q[i].x <= q[j].x ? q[i++] : q[j
++]);
5656     while (i <= m)
5757         p[k++] = q[i++];
5858     while (j <= r)
5959         p[k++] = q[j++];
6060     for (i = l; i <= r; ++i)
6161         q[i] = p[i];
6262 }
6363
6464 int main() {
6565     cin.tie(0)->sync_with_stdio(0);
6666     cin >> n >> m;
6767     for (int i = 1; i <= n; ++i) {

```

```

6868         cin >> a[i];
6969         S[a[i]].insert(i);
7070     }
7171     for (int i = 1; i <= n; ++i) {
7272         int pr = 0;
7373         for (int x : S[i]) {
7474             add_e(pr, x, 1);
7575             pr = x;
7676         }
7777     }
7878     while (m--) {
7979         int op, x, y;
8080         cin >> op >> x >> y;
8181         if (op == 1) {
8282             if (a[x] == y)
8383                 continue;
8484             int c1 = a[x], c2 = y;
8585             auto it = S[c1].find(x);
8686             int pr = (it == S[c1].begin() ? 0 : *
prev(it));
8787             auto nit = next(it);
8888             int nx = (nit == S[c1].end() ? 0 : *nit)
;
8989             add_e(pr, x, -1);
9090             add_e(x, nx, -1);
9191             add_e(pr, nx, 1);
9292             S[c1].erase(it);
9393
9494             S[c2].insert(x);
9595             it = S[c2].find(x);
9696             pr = (it == S[c2].begin() ? 0 : *prev(it
));
9797             nit = next(it);
9898             nx = (nit == S[c2].end() ? 0 : *nit);
9999             add_e(pr, nx, -1);

```

100100	add_e(pr, x, 1);
101101	add_e(x, nx, 1);
102102	a[x] = y;
103103	} else {
104104	q[++tot] = {tot, 1, y, x, ++qc, 0};
105105	}
106106	}
107107	cdq(1, tot);
108108	for (int i = 1; i <= qc; ++i)
109109	cout << ans[i] << "\n";
110110	}

判断题

- 27. 该程序采用 CDQ 分治的方法，离线处理所有的修改与查询操作。()
 - A. 正确
 - B. 错误
- 28. 函数 ask(x) 查询的是树状数组中所有下标不大于 x 的位置的元素之和。()
 - A. 正确
 - B. 错误
- 29. 若查询区间 [1, r] 内某种形状恰好出现一次，则该形状对该区间记忆值的贡献为 0。()
 - A. 正确
 - B. 错误

单选题

- 30. 函数 add_e(u, v, w) 中 if (u && v) 的作用是 ()
 - A. 仅当 u 与 v 均非 0 时，才生成一个事件
 - B. 当 u 与 v 中至少一个为 0 时，生成一个事件
 - C. 当 u 与 v 相等时，生成一个事件
 - D. 当 u 与 v 均非 0 时，不生成事件
- 31. 该程序 CDQ 分治部分的时间复杂度约为 ()
 - A. $O(n+m)$
 - B. $O((n+m) \log(n+m))$
 - C. $O((n+m) \log^2(n+m))$
 - D. $O((n+m)^2)$
- 32. 当输入为

7 6
 1 2 3 1 3 2 1
 2 3 7

2 1 3

1 7 2

1 3 2

2 1 6

2 5 7

时，第一个查询的输出为（ ）

A. 3

B. 5

C. 7

D. 9

三、完善程序（单选题，每小题 3 分，共计 30 分）

(1)（矩阵变换）给出一个 N 行 M 列的矩阵 A ，满足：① $M > N$ ；②矩阵中每个数都是 $[0, N]$ 中的自然数；③每行中 $[1, N]$ 中每个自然数恰好出现一次（因此每行中 0 恰好出现 $M-N$ 次）；④每列中 $[1, N]$ 中每个自然数至多出现一次。现在要在每行中选取一个非零数，并把这个数之后的数都赋值为这个数，希望在操作后性质④仍然成立。若存在方案，输出每行选取的数（应为 1 到 N 的一个排列）；否则输出任意卖萌表情。上述参数满足 $N \leq 200, M < 400, T < 50$ 。

提示：该问题可转化为稳定婚姻问题——每行与每个数字作为两类参与者，行 i 按数字 x 在该行中出现的位置 $p[i][x]$ 从小到大排序，数字 v 按其在各行中位置从大到小排序；再用队列依次尝试匹配，发生冲突时按双方偏好决定替换谁。

```
0101 #include <bits/stdc++.h>
0202 using namespace std;
0303 const int E = 4e2 + 5, Q = 4e4 + 5;
0404 int n, m, a[E][E], p[E][E], pr[E][E], pn[E][E], rk[E
    ][E], rn[E][E], mt[E],
0505     mn[E], q[Q], ptr[E];
0606 int main() {
0707     ios::sync_with_stdio(0);
0808     cin.tie(0);
0909     int T;
1010     cin >> T;
1111     while (T--) {
1212         cin >> n >> m;
1313         for (int i = 1; i <= n; ++i)
1414             for (int j = 1; j <= m; ++j) {
1515                 cin >> a[i][j];
1616                 if (a[i][j])
1717                     p[i][a[i][j]] = j;
1818             }
1919         for (int i = 1; i <= n; ++i) {
2020             iota(pr[i] + 1, pr[i] + n + 1, 1);
2121             sort(pr[i] + 1, pr[i] + n + 1,
2222                 [&](int x, int y) { return
    ① _____; });
2323             pr[i][n + 1] = 0;
2424         }
```

```

2525     for (int v = 1; v <= n; ++v) {
2626         iota(pn[v] + 1, pn[v] + n + 1, 1);
2727         sort(pn[v] + 1, pn[v] + n + 1,
2828             [&](int i, int j) { return
②; });
2929         for (int k = 1; k <= n; ++k)
3030             ③;
3131     }
3232     fill(mt + 1, mt + n + 1, 0);
3333     fill(mn + 1, mn + n + 1, 0);
3434     fill(ptr + 1, ptr + n + 1, 1);
3535     int h = 0, t = 0, ok = 1;
3636     for (int i = 1; i <= n; ++i)
3737         q[t++] = i;
3838     while (h < t) {
3939         int i = q[h++], v = pr[i][ptr[i]++];
4040         if (!v) {
4141             ok = 0;
4242             break;
4343         }
4444         if (!mn[v]) {
4545             mn[v] = i;
4646             mt[i] = v;
4747         } else {
4848             int j = mn[v];
4949             if (④) {
5050                 mn[v] = i;
5151                 mt[i] = v;
5252                 mt[j] = 0;
5353                 ⑤;
5454             } else
5555                 q[t++] = i;
5656         }
5757     }
5858     if (!ok)

```


5959	cout << "(^o^)/\n";
6060	else
6161	for (int i = 1; i <= n; ++i)
6262	cout << mt[i] << " \n"[i == n];
6363	}
6464	}

33. ①处应填 ()
- A. return p[i][x] < p[i][y]; B. return p[i][x] > p[i][y];
- C. return p[i][x] == p[i][y]; D. return p[i][x] - p[i][y];
34. ②处应填 ()
- A. return p[i][v] < p[j][v]; B. return p[i][v] > p[j][v];
- C. return p[i][v] == p[j][v]; D. return p[i][v] - p[j][v];
35. ③处应填 ()
- A. rn[v][pn[v][k]] = k; B. rn[v][k] = pn[v][k];
- C. rn[pn[v][k]][v] = k; D. rn[v][pn[v][k]] = pn[v][k];
36. ④处应填 ()
- A. if (rn[v][i] < rn[v][j]) B. if (rn[v][i] > rn[v][j])
- C. if (rn[v][i] == rn[v][j]) D. if (rn[i][v] < rn[j][v])
37. ⑤处应填 ()
- A. q[t++] = i; B. q[t++] = j;
- C. q[h++] = j; D. q[t] = j;

(2) (树上咒语相撞) 给定一棵 n 个结点的树 (结点编号 1 到 n)，其中有 $n-1$ 条无向道路。问有多少个三元组 (u, v, w) 满足： u, v, w 互不相同，且存在从 u 到 w 与从 v 到 w 的两条路径，这两条路径不经过相同的道路 (即 w 是 u 到 v 路径上的中间点，且 u, v 位于 w 的不同分支)。同时，会不断在树中加入新边 (新边可能连接已存在的结点对，也可能连接同一个结点自身)，每加入一条新边后，需要输出当前满足条件的三元组个数。上述参数满足 $n, q \leq 10^5$ 。

提示：三元组计数可按 w 分类： u, v 必须落在 w 的不同的相邻分支中。加边后，环上的点会被并查集合并成一个分量，维护每个分量的大小 z 与“锚点子树”大小 u ，并利用原树信息 s 更新答案。

```

0101 #include <bits/stdc++.h>
0202 using namespace std;
0303 typedef long long ll;
0404 const int E = 1e5 + 5;
0505 int n, hd[E], to[E * 2], ne[E * 2], cnt, fa[E], dep[
    E], sz[E], f[E];
0606 ll z[E], s[E], u[E], ans;
0707 void add(int x, int y) { to[++cnt] = y, ne[cnt] = hd
    [x], hd[x] = cnt; }
0808 void dfs(int x, int p) {
0909     fa[x] = p, dep[x] = dep[p] + 1, sz[x] = 1;
1010     for (int i = hd[x]; i; i = ne[i])
1111         if (to[i] != p)
1212             dfs(to[i], x), ①;
1313 }
1414 int find(int x) { return f[x] == x ? x : f[x] = find
    (f[x]); }
1515 void merge(int x, int y) {
1616     x = find(x), y = find(y);
1717     if (x == y)
1818         return;
1919     ll cx = z[x], cy = z[y], tx = u[x], ty = n - tx;
2020     ans -= cx * (cx - 1) * (cx - 2) + 2 * cx * (cx -
    1) * (n - cx) + cx * ((n - cx) * (n - cx) - s[x
    ]]);
2121     ans -= cy * (cy - 1) * (cy - 2) + 2 * cy * (cy -

```

```

1) * (n - cy) + cy * ((n - cy) * (n - cy) - s[y
]);
2222 s[y] = s[y] - tx * tx + s[x] - ty * ty;
2323 ⑤;
2424 f[x] = y;
2525 cx = z[y];
2626 ans += cx * (cx - 1) * (cx - 2) + 2 * cx * (cx -
1) * (n - cx) + cx * ((n - cx) * (n - cx) - s[y
]);
2727}
2828int main() {
2929 ios::sync_with_stdio(0);
3030 cin.tie(0);
3131 cin >> n;
3232 for (int i = 1; i < n; ++i) {
3333 int x, y;
3434 cin >> x >> y;
3535 add(x, y);
3636 add(y, x);
3737 }
3838 dfs(1, 0);
3939 for (int i = 1; i <= n; ++i)
4040 f[i] = i, z[i] = 1, ③;
4141 for (int i = 1; i <= n; ++i) {
4242 ll t = 0;
4343 for (int j = hd[i]; j; j = ne[j])
4444 if (to[j] != fa[i])
4545 t += ②;
4646 if (fa[i])
4747 t += 1ll * (n - sz[i]) * (n - sz[i]);
4848 s[i] = t;
4949 }
5050 for (int i = 1; i <= n; ++i) {
5151 ll c = z[i];
5252 ans += c * (c - 1) * (c - 2) + 2 * c * (c -

```

	1) * (n - c) + c * ((n - c) * (n - c) - s[i]);
5353	}
5454	cout << ans << '\n';
5555	int q;
5656	cin >> q;
5757	while (q--) {
5858	int x, y;
5959	cin >> x >> y;
6060	x = find(x), y = find(y);
6161	while (x != y) {
6262	if (____④____)
6363	swap(x, y);
6464	int p = find(fa[x]);
6565	merge(x, p);
6666	x = find(x);
6767	}
6868	cout << ans << '\n';
6969	}
7070	}

38. ①处应填 ()
A. sz[x] += sz[x]; B. sz[x] += sz[to[i]];
C. sz[to[i]] += sz[x]; D. sz[x] = sz[to[i]];
39. ②处应填 ()
A. t += sz[to[j]] * sz[to[j]]; B. t += 111 * sz[to[j]] * sz[to[j]];
C. t += 111 * sz[to[j]] * sz[to[i]]; D. t += 111 * sz[i] * sz[to[j]];
40. ③处应填 ()
A. u[i] = i; B. u[i] = sz[i];
C. u[i] = dep[i]; D. u[i] = n;
41. ④处应填 ()
A. if (dep[x] < dep[y]) swap(x, y); B. if (dep[x] > dep[y]) swap(x, y);
C. if (sz[x] < sz[y]) swap(x, y); D. if (fa[x] < fa[y]) swap(x, y);
42. ⑤处应填 ()
A. z[y] += z[x]; B. z[x] += z[y];
C. z[y] = z[x]; D. f[y] = x;